



Python

Install

See:

<https://docs.anaconda.com/anaconda/install/>

<https://docs.anaconda.com/anaconda/user-guide/getting-started/>

Install also:

Notebook, Spyder, Orange3

Definition

- Python is an easy to learn, powerful programming language.
- It has efficient high-level data structures and a simple but effective approach to object-oriented programming.
- Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms.

More Info

- Python is cross platform (for many platforms).
- The standard library is huge, very useful, and is very well documented.
 - <http://docs.python.org/library/>
- Many companies make significant use of it
 - Google uses it for a lot of their internal software projects

More Info

- Created by [Guido van Rossum](#) and first released in 1991, Python's design philosophy emphasizes [code readability](#) with its notable use of [significant whitespace](#)
- Python is [dynamically typed](#) and [garbage-collected](#). It supports multiple [programming paradigms](#), including [structured](#) (particularly, [procedural](#)), object-oriented, and [functional programming](#)

Why is it called Python?

When he began implementing Python, Guido van Rossum was also reading the published scripts from [“Monty Python’s Flying Circus”](#), a BBC comedy series from the 1970s.



2- Basic data types

Numeric data types

- Computers “simulate” the real number system.
- Two numeric data types:
 - Integer (`int`), e.g., 10, 0, -9999
 - Floating-point number (`float`), e.g., 1.1, 0., -3333.33
- `int` and `float` are two different *data types*.
- A floating-point number can be represented by an exponent component, e.g., -3.33333×10^3 (type -3.33333e3 in Python)
- Inside the computer, integers and floating point are represented quite differently.

String

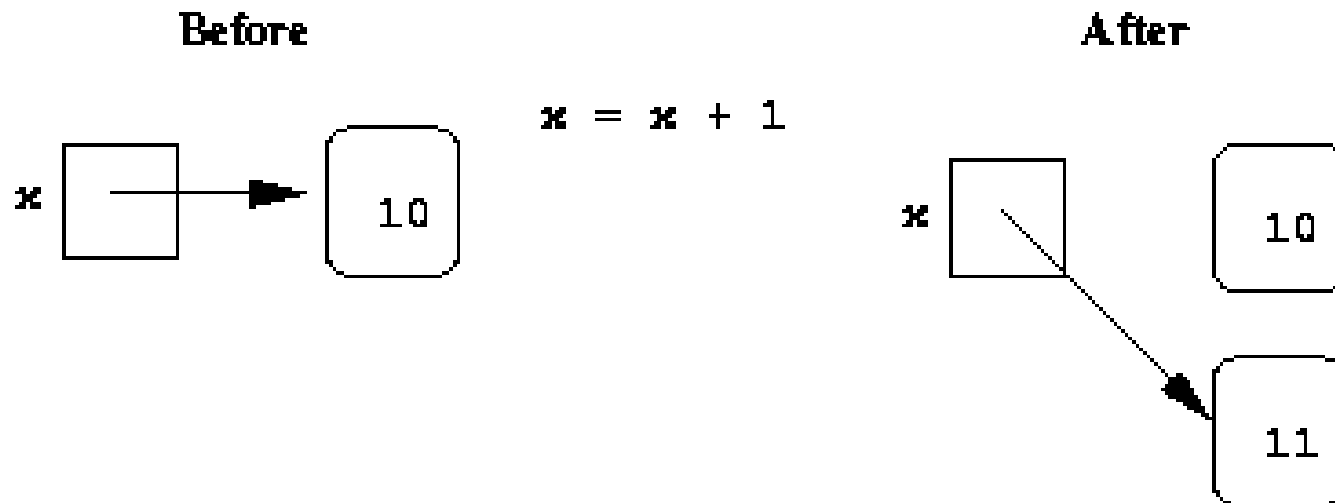
- Strings in Python can be expressed inside double quotes or single quotes.
 - A string can be empty.
- Strings in Python are represented by UTF-8 using 8 to 32 bits to represent a character.
 - The 8-bit representation is the same as the ASCII.
- The `ord` function returns the numeric (ordinal) code of a single character, e.g., `ord('A')` is 65.
- The `chr` function converts a numeric code to the corresponding character, e.g., `chr(65)` is 'A'.

Variables and identifiers

- Besides literals, a variable can be used to hold a value.
 - A variable is a name (identifier) that is associated with a value
- An identifier is also used to name functions and modules.
- Every identifier must begin with a letter or underscore (“_”), followed by any sequence of letters, digits, or underscores.
 - Note that “—”, “.” and many other symbols are not allowed.
 - Identifiers are case sensitive.
 - Identifiers cannot be Python’s keywords.

Behind the scene

- Python does not recycle these memory locations.
 - Integer values are immutable.
- Assigning a variable is more like putting a “sticky note” on a value and saying, “this is x”.
- The memory for the old value will be “released” automatically (i.e., *garbage collection*).



Python built-in numeric operations

Operation	Result
<code>x + y</code>	sum of x and y
<code>x - y</code>	difference of x and y
<code>x * y</code>	product of x and y
<code>x / y</code>	quotient of x and y
<code>x // y</code>	floored quotient of x and y
<code>x % y</code>	remainder of <code>x / y</code>
<code>-x</code>	x negated
<code>+x</code>	x unchanged
<code>abs(x)</code>	absolute value or magnitude of x
<code>int(x)</code>	x converted to integer
<code>float(x)</code>	x converted to floating point
<code>complex(re, im)</code>	a complex number with real part <i>re</i> , imaginary part <i>im</i> . <i>im</i> defaults to zero.
<code>c.conjugate()</code>	conjugate of the complex number c
<code>divmod(x, y)</code>	the pair <code>(x // y, x % y)</code>
<code>pow(x, y)</code>	x to the power y
<code>x ** y</code>	x to the power y

Source: Charles Dierbach. 2013. Introduction to Computer Science Using Python. Wiley.

Operator precedence and associativity

The unary operators `**` and `-` (negation) have higher precedence than the four operators `+`, `-`, `*`, `/`.

For operators of the same precedence, the associativity determines the order of their operations.

Operator	Associativity
<code>**</code> (exponentiation)	right-to-left
<code>-</code> (negation)	left-to-right
<code>*</code> (mult), <code>/</code> (div), <code>//</code> (truncating div), <code>%</code> (modulo)	left-to-right
<code>+</code> (addition), <code>-</code> (subtraction)	left-to-right

3. Decision Structures

Two-Way Decisions

- In Python, a two-way decision can be implemented by attaching an `else` clause onto an `if` clause.

- This is called an `if-else` statement:

```
if <condition>:  
    <statements>  
else:  
    <statements>
```

- E.g.,

```
if 1 <= n <= 10000:  
    print("Your input number is ", str(n)+".")  
else:  
    print("Your input must be between 1 and 1000.")
```

Multi-Way Decisions

- Use nested `if-else` statement to implement multi-way decisions.

- `if <condition1>:`
 `<case1 statements>`
- `else:`
 `if <condition2>:`
 `<case2 statements>`
 `else:`
 `if <condition3>:`
 `<case3 statements>`
 `else:`
 `<default statements>`

Multi-Way Decisions using `elif`

- In Python, `else-if` is combined into `elif`:

```
if <condition1>:  
    <case1 statements>  
elif <condition2>:  
    <case2 statements>  
elif <condition3>:  
    <case3 statements>  
else:  
    <default statements>
```

The `else` is optional. If there is no `else`, it is possible that no indented block would be executed.

while statement

```
while <condition>:  
    <body>
```

`condition` is a Boolean expression, just like in `if` statements. The body is a sequence of one or more statements.

Semantically, the body of the loop executes repeatedly as long as the condition remains true. When the condition is false, the loop terminates.

4. Sequence data type

List

A **list** is a linear data structure, thus its elements have a linear ordering.

- A list of names, numbers, student records, grocery, etc

You can create a list in Python by

- `myList = [1, 2, 3, 4]`
- `myGrades = ["A+", "A", "B+", "B"]`
- `myMenu = ["Sausage", "egg", "bread", "potato"]`
- `myMix = [1, "Spam ", 4, "U"]`

A **list** in Python is a mutable, linear data structure of variable length, allowing mixed-type elements.

- A list can be empty, e.g., `empty_list = []`.

Accessing and modifying a list

You can use indices or methods or operators or built-in functions to manipulate a list.

- The index starts from 0.

Operation	fruit = ['banana', 'apple', 'cherry']	
Replace	fruit[2] = 'coconut'	['banana', 'apple', 'coconut']
Delete	del fruit[1]	['banana', 'cherry']
Insert	fruit.insert(2, 'pear')	['banana', 'apple', 'pear', 'cherry']
Append	fruit.append('peach')	['banana', 'apple', 'cherry', 'peach']
Sort	fruit.sort()	['apple', 'banana', 'cherry']
Reverse	fruit.reverse()	['cherry', 'banana', 'apple']

Tuples

A **tuple** is an *immutable* linear data structure.

- The elements in a tuple cannot be modified.

You can create a tuple in Python by

- `myList = (1, 2, 3, 4)`
- `myGrades = ("A+", "A", "B+", "B")`
- `myMenu = ("Sausage", "egg",
"bread", "potato")`
- `myMix = (1, "Spam ", 4, "U")`
- `myEmptiness = ()`

Tuples of one element must include a comma following the element, e.g., `(1, )` (instead of `(1)`).

Sequences

- A **sequence** in Python is a linearly ordered set of elements accessed by an index number.
 - Lists, tuples, and strings are all sequences.
 - Strings, like tuples, are immutable.
- A string is a sequence of characters.

H	e	l	l	o		W	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11

String slicing

We can also access a contiguous sequence of characters, called a *substring*, through a process called *slicing*.

Slicing: `<string>[<start>:<end>]`

- `start` and `end` should both be integers.
- The slice contains the substring beginning at position `start` and runs up to **but doesn't include** the position `end`.
- The defaults are the beginning and the end.

Common operations for sequences

Operation		String s = 'hello' w = '!'	Tuple s = (1,2,3,4) w = (5,6)	List s = [1,2,3,4] w = [5,6]
Length	len(s)	5	4	4
Select	s[0]	'h'	1	1
Slice	s[1:4]	'ell'	(2, 3, 4)	[2, 3, 4]
	s[1:]	'ello'	(2, 3, 4)	[2, 3, 4]
Count	s.count('e')	1	0	0
	s.count(4)	error	1	1
Index	s.index('e')	1	--	--
	s.index(3)	--	2	2
Membership	'h' in s	True	False	False
Concatenation	s + w	'hello!'	(1, 2, 3, 4, 5, 6)	[1, 2, 3, 4, 5, 6]
Minimum Value	min(s)	'e'	1	1
Maximum Value	max(s)	'o'	4	4
Sum	sum(s)	error	10	10

Source: Charles Dierbach. 2013. Introduction to Computer Science Using Python. Wiley.

For statement (iterating over sequences)

- A **for statement** is an iterative control statement that iterates once for each element in a specified sequence of elements.
- The `for` statement allows us to iterate through a sequence of values.

```
for <var> in <sequence>:  
    <body>
```

- The loop index variable `var` takes on each successive value in the sequence, and the statements in the body of the loop are executed once for each value.

The range function

A built-in **range** function can be used for generating a sequence of integers that a for loop can iterate over.

- Rather than being a function, range is actually an immutable sequence type.

Usage: `range(start, stop[, step])`

- The arguments must be integers.
- If the *step* argument is omitted, it defaults to 1.
- If the *start* argument is omitted, it defaults to 0.
- Try `range(1, 10)`, `range(-1, 5)`, `range(-1, -10, -1)`.

Actually, range does not create a sequence of integers.

- It creates a *generator function* able to produce each next item of the sequence when needed.
- This saves memory, especially for long lists.

String methods

For Python 3.0:

<https://docs.python.org/3/library/stdtypes.html>

- `str.capitalize()`
- `str.casefold()`
- `str.center(width[, fillchar])`
- `str.count(sub[, start[, end]])`
- `...`
- `str.title()`
- `str.translate(map)`
- `str.upper()`
- `str.zfill(width)`

The string format () method

String formatting syntax:

- `<template-string>.format(<values>)`
- E.g., `"The value of pi is {0:0.4f}".format(math.pi)`

Curly braces(`{ }`) inside the `template-string` marks “slots” into which the provided values are inserted.

- `{<index>:<format-specifier>}`, e.g., `{0:0.4f}`
- `index`: indicate which parameter to insert here
- One form of specifiers is `<width>.<precision><type>`.
 - `width`: the number of spaces for printing the parameter
 - `precision`: the number of decimal places
 - `f`: will print out 0 to fill up the number of decimal places.

6. Dictionaries and sets

Storing students' records

Problem: How to store student's data in a program after reading the data from a file?

A possible way is to store the data in a list.

However, a list can be indexed only by non-negative integers, but ours should be indexed by names.

- Map the names to a set of integers.
- Use the integers as indices for accessing the list.

A much better solution is to use a Python dictionary that maps the set of names (keys) directly to a set of values.

Python dictionary

A dictionary is a mutable, associative data structure of variable length.

- The key can be of any immutable type.
- The values can be of any type and are unordered.

An example:

- `dicta = {'rocky': 'CHANG Kow Chuen', ('a', 'b', 'c'): [1, 2, 3], 1: 2}`
- `divmod(x,y)` returns a tuple.
- Get the value using its index, e.g., `dicta["rocky"]`.

Dictionary operators

Operation	Results
<code>dict()</code>	Creates a new, empty dictionary
<code>dict(s)</code>	Creates a new dictionary with key values and their associated values from sequence <code>s</code> , for example, <pre>fruit_prices = dict(fruit_data)</pre> where <code>fruit_data</code> is (possibly read from a file): <pre>[['apples', .66], ..., ['bananas', .49]]</pre>
<code>len(d)</code>	Length (num of key/value pairs) of dictionary <code>d</code> .
<code>d[key] = value</code>	Sets the associated value for <code>key</code> to <code>value</code> , used to either add a new key/value pair, or replace the value of an existing key/value pair.
<code>del d[key]</code>	Remove key and associated value from dictionary <code>d</code> .
<code>key in d</code>	True if key value <code>key</code> exists in dictionary <code>d</code> , otherwise returns False.

Source: Charles Dierbach. 2013. Introduction to Computer Science Using Python. Wiley.

Dictionary methods

Python provides a number of other methods that iterate through the elements in a dictionary.

- `items()`: all the key-values pairs as a list of tuples
- `keys()`: all the keys as a list
- `values()`: all the values as a list

The objects returned by `dict.keys()`, `dict.values()` and `dict.items()` are *view objects*.

- They provide a dynamic view on the dictionary's entries, which means that when the dictionary changes, the view reflects these changes.
- They are iterable, therefore can be used in a loop.

Sets in Python

A set is collection of non-mutable objects which are elements of the set.

Set is mutable.

The elements could be of any types.

There is no order in a set.

There are no duplicate elements in a set.

`{}` is an empty set.

Set operators

Set operator		Set A = {1,2,3}	Set B = {3,4,5,6}	
membership		<code>1 in A</code>	True	<i>True if 1 is a member of set</i>
add		<code>A.add(4)</code>	{1,2,3,4}	<i>Adds new member to set</i>
remove		<code>A.remove(2)</code>	{1,3}	<i>Removes member from set</i>
union		<code>A B</code>	{1,2,3,4,5,6}	<i>Set of elements in either set A or set B</i>
intersection		<code>A & B</code>	{3}	<i>Set of elements in both set A and set B</i>
difference		<code>A - B</code>	{1,2}	<i>Set of elements in set A, but not set B</i>
symmetric difference		<code>A ^ B</code>	{1,2,4,5,6}	<i>Set of elements in set A or set B, but not both</i>
size		<code>len(A)</code>	3	<i>Number of elements in set (general sequence operation)</i>

Source: Charles Dierbach. 2013. Introduction to Computer Science Using Python. Wiley.

6. Functions

Function definition

The part of the program that creates a function is called a *function definition*.

```
def <name> (<formal-parameters>) :  
    <body>
```

E.g.,

```
def avg (n1, n2, n3) :  
    print ( (n1+n2+n3) / 3)
```

- where `n1`, `n2`, `n3` are *formal parameters*.

A function is called/invoked by `<name> (<actual-parameters>)`

- E.g., `avg (1, 2, 3)`, where 1, 2 and 3 are *actual parameters*.

Functions must be defined/imported before it is called.

(Non-)Value-returning functions

The three arguments are passed to the three parameters according to their positions.

A **value-returning function** is a program routine called for its return value.

- The *return statement* of the form `return expr`, where `expr` may be any expression.

A **non-value-returning function** is called not for a returned value, but for its *side effects*.

- A **side effect** is an action other than returning a function value, such as displaying output on the screen.
- There is no return statement.

Positional and keyword arguments

A **positional argument** is an argument that is assigned to a particular parameter based on its position in the argument list.

A **keyword argument** is an argument that is specified by parameter name.

- E.g.,

```
def printNumbers(n1, n2, n3):  
    print(n1, n2, n3)  
  
m1, m2, m3 = 10, 20, 30  
printNumbers(m1, m2, m3)  
printNumbers(n1=m2, n2=m3, n3=m1)
```


Local scope and local variables

Scope: places in a program where the parameters and variables are referenced.

A **local variable** is a variable that is only accessible from within a given function. Such variables are said to have **local scope**.

Formal parameters also have local scope.

Local variables are automatically created (allocated memory) when a function is called, and destroyed (deallocated) when the function terminates.

Global variables and global scope

A **global variable** is a variable that is defined outside of any function definition. Such variables are said to have **global scope**.

For example.

```
max_count = 1000    # a global variable
def funA(count):
    if count < max_count:
        ...
def funB(count):
    for i in range(max_count):
        ...
```

The use of global variables is generally considered to be bad programming style.

7. Modules

Where do the modules come from?

- The term “module” refers to the design and/or implementation of specific functionality to be incorporated into a program.

- Modular-II

```
MODULE Hello;  
FROM STextIO IMPORT WriteString;  
BEGIN  
    WriteString("Hello World!");  
END Hello.
```

A Python module is a file containing Python definitions and statements.

- By convention, modules are named using all lower case letters and optional underscore characters.

Interface, client and docstring

A module's *interface* is a specification of what it provides and how it is to be used.

Any program code making use of a given module is called a *client* of the module.

A *docstring* is a string literal denoted by triple quotes used in Python for providing the specification of certain program elements.

Documentation String (docstring)

([http://legacy.python.org/dev/peps/pep-0008/#documentation-](http://legacy.python.org/dev/peps/pep-0008/#documentation-strings)

A *docstring* is a string literal denoted by triple quotes given as the first line of certain program elements.

These additional lines must be indented at the same level.

The docstring for a module is simply the first block of quoted text to appear in the module.

Write docstrings for all public modules, functions, classes, and methods.

Convention:

```
"""Return a foobang
```

```
Optional plotz says to frobnicate the bizbaz  
first. """
```

Running a module as main or imported module

Before executing the code, the Python interpreter will define a few special variables, e.g., `__name__`.

If the python interpreter is running that module (the source file) as the main program, it sets the special `__name__` variable to have a value `"__main__"`.

If this file is being imported from another module, `__name__` will be set to the module's name.

`test()` in `graphics.py` will be executed only when the module is run as main:

```
if __name__ == "__main__":  
    test()
```

See https://docs.python.org/3/library/__main__.html

Searching for imported modules

Each imported module of a Python program needs to be located and loaded into memory.

Python first searches for modules in the current directory. If the module is not found, it searches the directories specified in the PYTHONPATH environment variable.

If the module is still not found, a Python installation-specific path is searched (e.g., `C:\Python32\Lib`).

If the program still does not find the module, an `ImportError` exception is reported.

The main and imported modules

The main module may *import* any number of other modules (and each of those modules import other modules, etc.).

Main modules are not meant to be imported into other modules.

The statements of imported modules are executed only once, the first time that the module is imported.

- When a module is loaded, a compiled version of the module with file extension `.pyc` is automatically produced.

Imported modules:

- Standard library (usually written in C) or
- Other modules (usually written in Python)

Global and local variables

A *local variable* is a variable that is only accessible from within the function it resides. Such variables are said to have *local scope*.

A *global variable* is a variable defined outside of any function definition. Such variables are said to have *global scope*.

The use of global variables is considered bad programming practice.

Scope resolution

Scope of a name (identifier) is the set of program statements over which it can be referred to.

The scope of a name depends on where it is created and whether the name is used somewhere else.

- Local and global scopes

When encountering a name, Python interpreter will search it in the order below:

- Local
- Enclosing (don't worry about it)
- Global
- Built-in

The local assignment rule

If anywhere in a function an assignment is made, then that assignment is assumed to create a name *only in the presently active namespace*.

If, within a function, a variable is declared to be a global variable using the `global` statement, then Python will not create a local name in the namespace for that variable.

Modules and Namespaces

A *namespace* is a container that provides a named context for a set of identifiers (for variables, functions, and constants).

A *name clash* is when two otherwise distinct entities with the same name become part of the same scope.

In Python, each module has its own namespace.

- This includes the names of all items in the module.

The *fully qualified* name of each identifier in a module is of the form *modulename.identifier*.

`math.sqrt()` in a (e.g., `math`) module is used when the module is imported.

A module's/global namespace

In Python, the main module of any program is the first (“top-level”) module executed.

When working interactively in the Python shell, the Python interpreter functions as the main module, containing the *global namespace*.

- The namespace is reset every time the interpreter is started.

Six names are always there: `'__builtins__'`,
`'__doc__'`, `'__loader__'`, `'__name__'`,
`'__package__'`, `'__spec__'`

Importing modules

When using `import module_name`, the namespace of the imported module becomes *available to*, but not *part of*, the importing module.

- The imported identifiers must be referenced fully qualified.

When using `from module_name import something`, the imported module's namespace *becomes part of* the importing module's namespace.

- The imported identifiers are referenced without being fully qualified.

From-import

- Three possible from-imports:

- `from module_name import func1, func2`
- `from module_name import func1 as new_func1`
- `from module_name import *`

- For `import *`,

- All of the identifiers are imported, except for those that begin with two underscore characters, which are meant to be *private* in the module.

- In Python, all the variables in a module are “public,” with the convention that variables beginning with two underscores are intended to be private.

Local, Global, and Built-in Namespaces

During a Python program's execution, there are as many as three namespaces:

- The *built-in namespace* contains the names of all the built-in functions, constants, and so on, in Python.
- The *global namespace* contains the identifiers of the currently executing module.
- The *local namespace* is the namespace of the currently executing function (if any).

`locals ()` and `globals ()` built-in functions

`locals ()` : Update and return a dictionary representing the current local symbol table.

`globals ()` : Return a dictionary representing the current global symbol table.

The builtins module

When the Python interpreter starts up, it loads two default modules without requiring an import:

`__main__` and `__builtins__`.

The `__builtins__` module provides the built-in functions and built-in constants.

More details on this module:

<https://docs.python.org/3/library/builtins.html>

8. Classes

Methods in Classes

- Define a method in a class by including function definitions within the scope of the class block
- There must be a special first argument **self** in all of method definitions which gets bound to the calling instance
- There is usually a special method called **__init__** in most classes

A Simple Class def: Student

```
class student:
    """A class representing a student """
    def __init__(self , n, a):
        self.full_name = n
        self.age = a
    def get_age(self):    #Method
        return self.age
```

- Define class:
 - Class name, begin with capital letter, by convention
 - object: class based on (Python built-in type)
- Define a method
 - Like defining a function
 - Must **have a special first parameter**, self, which provides way for a method to refer to object itself

Instantiating Objects with ‘__init__’

- `__init__` is the default constructor
- `__init__` serves as a constructor for the class. Usually does some initialization work
- An `__init__` method can take any number of arguments
- However, the first argument `self` in the definition of `__init__` is special

Self

- The first argument of every method is a reference to the current instance of the class
- By convention, we name this argument **self**
- In `__init__`, `self` refers to the object currently being created; so, in other class methods, it refers to the instance whose method was called
- Similar to the keyword `this` in Java or C++
- But Python uses `self` more often than Java uses `this`
- You **do not** give a value for this parameter(`self`) when you call the method, Python will provide it.

Continue...

...Continue

- Although you must specify `self` explicitly when defining the method, you don't include it when calling the method.
- Python passes it for you automatically

Defining a method:

Calling a method:

(this code inside a class definition.)

```
def get_age(self, num):  
    self.age = num
```

```
>>> x.get_age(23)
```

Public, Protected and Private Data

- If an identifier doesn't start with an underscore character "_" it can be accessed from outside, i.e. the value can be read and changed
- Data can be protected by making members private or protected. Instance variable names starting with two underscore characters cannot be accessed from outside of the class.
- At least not directly, but they can be accessed through private name mangling.
- That means, private data `__A` can be accessed by the following name construct: ***instance_name.__classname__A***

-
- If an identifier is only preceded by one underscore character, it is a protected member.
 - Protected members can be accessed like public members from outside of class

Example:

```
class Encapsulation(object):  
    def __init__(self, a, b, c):  
        self.public = a  
        self._protected = b  
        self.__private = c
```

The following interactive sessions shows the behavior of public, protected and private members:

```
>>> x = Encapsulation(11,13,17)
```

```
>>> x.public
```

```
11
```

```
>>> x._protected
```

```
13
```

```
>>> x._protected = 23
```

```
>>> x._protected
```

```
23
```

```
>>> x.__private
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
AttributeError: 'Encapsulation' object has no attribute '__private'
```

```
>>>
```

The following table shows the different behavior Public, Protected and Private Data

Name	Notation	Behavior
name	Public	Can be accessed from inside and outside
_name	Protected	Like a public member, but they shouldn't be directly accessed from outside
__name	Private	Can't be seen and accessed from outside

9. Files

File (object)

A *file* is a sequence of data that is stored in secondary memory (disk drive).

Two types of files: *text file* and *binary file*

- A text file contains characters, structured as lines of text.
- A binary file is a file formatted in a way that only a computer program can read.

A text file usually contains more than one line of text. Lines of text are separated with a special character, the *newline* character.

Reading/writing and saving a file

Load (part of) the file into the main memory.

Read the file from the memory.

Write the file to the memory.

Saving will write the file in the memory to a secondary storage.

Three file read methods

`<filevar>.read()` – returns the entire remaining contents of the file as a single (possibly large, multi-line) string.

`<filevar>.readline()` – returns the next line of the file. This is all text up to *and including* the next newline character.

`<filevar>.readlines()` – returns a list of the remaining lines in the file. Each list item is a single line including the newline characters.

Write and append methods

Opening a file for writing prepares the file to receive data.

If you open an existing file for writing, you wipe out the file's contents. If the named file does not exist, a new one is created.

It is important to close a file that is written to, otherwise the tail end of the file may not be written to the file.